Day 1:

What is data?

- Collection of facts
- is information
- Makes you wanna make a decision.
- Data make you to take action.

Types of data:

Structured data

- Data Stored in table format with rows and Columns
- Excel Spreadsheet

Unstructured data

- Social media
- PDF Documents
- Music
- Video / Adverts.
- Phone calls
- No Consistent format

Application form

OrderID	Product	Price	Qty

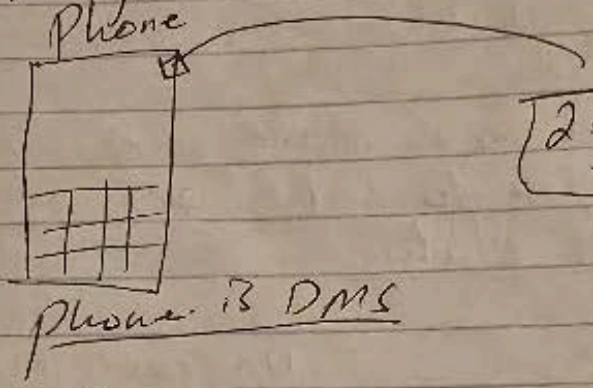
Name	Phone	Lat

Rational database (RDB)

- A Storage place for Structured data.
- A database is where we store the data
- is a tool that allows us to interact with data.

Database Management Systems (DMS)

↳ A System we use to access the data.



Phone is DMS

functions

- Control access
- Delete music
- Retrieve (play music)
- Access data.

Aws
GCP
AZURE
ORACLE

Second example

DMS
Computer

- Retrieve
- update
- Access
- Add Stuff

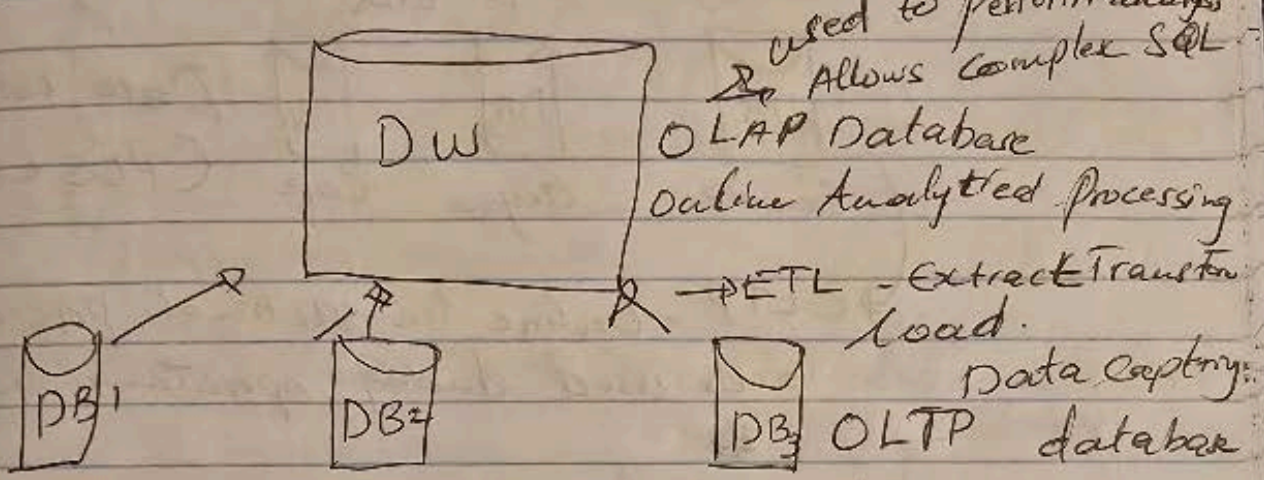
(RDB)
Memory Stick

* Movies
* files
* Music
* documents

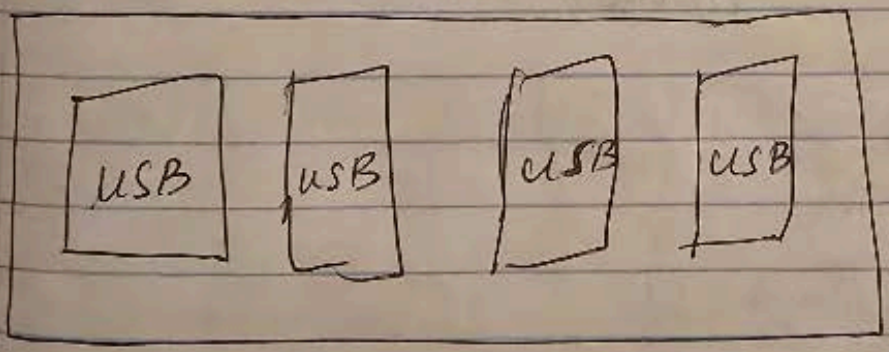
Day 2.

Datawarehouse (DW).

* is a central place where we store multiple databases.

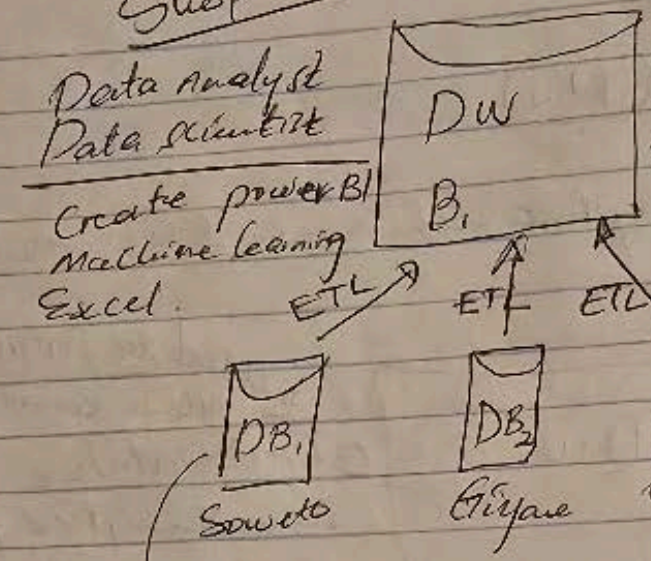


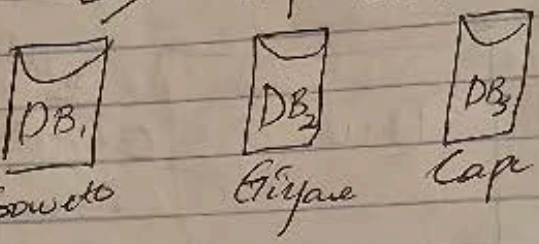
AWS.



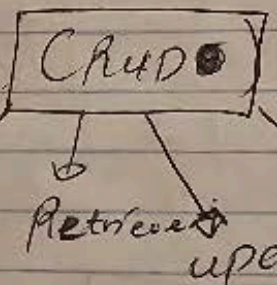
GCP - Google Cloud platform.
AWS - Amazon web Service
AZURE -
ORACLE -
META -

Shoprite Example



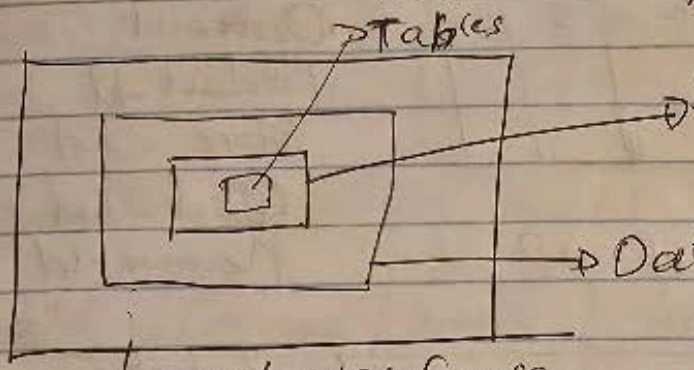


↳ OLTP - Online transactional processing data
↳ used during operational hours (Bu
hou

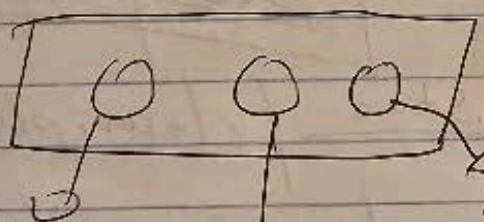


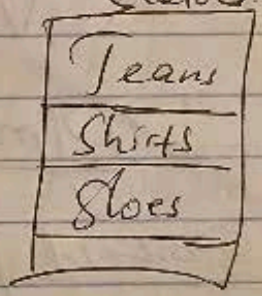
Day 3:

Datawarehouse - is a Central Data Store.
- houses multiple databases.



Schema - helps to seperate different tables





Tables

Dimension tables
- is a table that stores descriptive information

Product id	Product name	Category	Sub category
123	Apple	fruit	fresh
124	Banana	fruit	wild
125	Orange	fruit	Citrus

Fact table
- more quantitative
- used for analysis.
↳ Stores transactional information.
↳ Records what happened.
↳ Stores numeric values you want to measure.

Answers
what was sold?
How much was sold?

Fact table → Store details in numbers or numerical table

Order-id	Product-id	store-id	Qtd	Price
346	125	12	3	R40

Fact table

Sale_Id
Date_id
Product_Id
Store_Id
Qtd_Id
Revenue_Id

Orange previous page Sandton → link up.

Fact table

Order id	Product id	Store id	Qtd
346	125	12	3

Dimension Store | descriptive data.

Store-Id	Store-name
11	Soweto
12	Sandton
10	Braam

A primary key - is a column that uniquely identifies each row in a table

A foreign key - is a column that link one table to another.

Primary SA Citizens

Id-no	Names
96...	Khutso
96....	thabiso
41...	zodwa

Porsche-Sales

order-id	Car name	n
111	GT911	

Porsche Sales

Erased..

Order-Id	Car name	model	Year	Customer name	ID no
111	Porsche	GT911	2012		96....
112	BMW	M4	2024		96....

(foreign key)

→ Primary

Customer table

Cust-ID	Name
111	Rofhiwa
112	Jeff
113	Mark
114	Elon

Orders

Foreign key

order-ID	Cust-ID	Cust-name
232	111	Rofhiwa
124	111	Rofhiwa
432	114	Elon
346	112	Jeff

↳ Cant repeat

Can Repeat

Assignment:

Handwritten:

Datalake
Datamart
foreign key

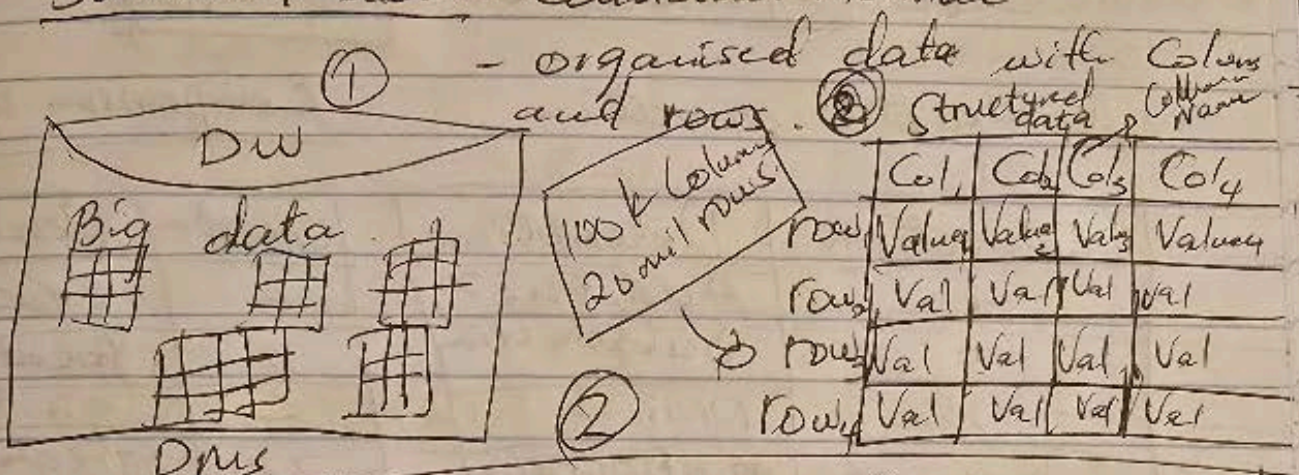
Primary key
Database
Datawarehouse
Schema.

* AI, * Prompt Engineering, * Snowflake
* Databricks, Microsoft Sequel Server.
* Github, Git, Gitlab, Version Control
* Big query, Power BI, Looker Studio

Day 4 Week 2.

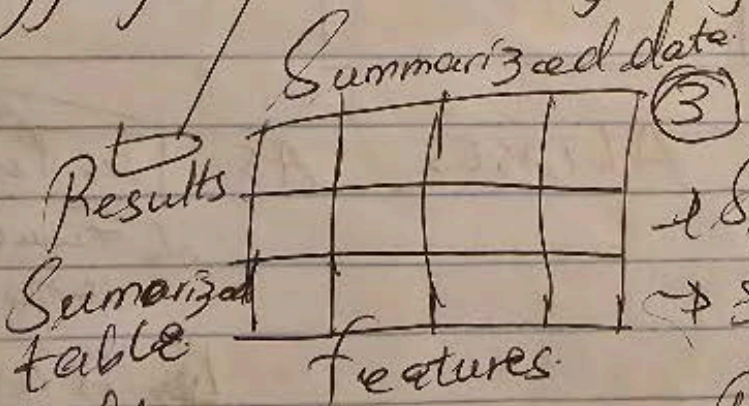
Data is information
↳ we use for decision making.

Structured data Consistent format
- organised data with Colum and rows.



*Uses of SQL

*Data cleaning, Data exploration, Data inspection, Data processing, Data wrangling, Data aggregation, Data joining, Data filtering



*After results, you can use Power BI to build dashboards. (4)

Excel (4)

Python.

Pandas - Library.

PySpark

SQL Operations in coding

Arithmetic operation

Operation	Description
+	Addition
-	Substraction
/	Division
*	multiplication.

Comparison Operator

Operation	Description
=	Equals to
>	Greater than
<	less than
>=	Greater or Equ
<=	less than or Equa

logical operators.

operator	Description
AND	
BETWEEN	
IN	
NOT	
OR	

ALIASES AS — as function

1. Function used to rename
2. is a function to give something a Nickname

Day 5 Week 2.

SQL - Structured Query Language

SELECT STATEMENT

↳ Allows us to SELECT specific columns in our data.

Table-name

	Col1	Col2	Col3	Col4
Row1				
Row2				
Row3				
Row4				

Original

FROM Statement

↳ Allows us to specify the name of the table we are selecting the data from.

```
SELECT Col1,
       Col2
       Col3
FROM Table_name
```

Col1	Col2	Col3

SELECT *

FROM TABLE name

↳ Allows us to select all the columns in our table.

Students_table

Student_ID	Name	Subject	Marks
101	Rofhiwa	Maths	99
102	Mbali	Maths	20
103	Sipho	Physics	60
104	Tumi	GEO	85

```
SELECT
    Name,
    Subject.
FROM Student_table;
```

Why comma.
→ we separate the Column names with a comma within the SELECT Statement.
* A Comma Seperates different Columns names.

Name	Subject
Rofhiwa	Maths
Mbali	Maths
Sipho	Physics
Tumi	GEO

```
SELECT *
FROM Students_table;
```

* is an automatic way of selecting all the columns in your table.

SELECT * will return the same table.

Student_ID	Name	Subject	Marks
101	Rofhiwa	maths	99
102	Mbali	Maths	20
103	Sipho	Physics	60
104	Tumi	GEO	85

→ Closing statement Shows the script ends here.

SELECT DISTINCT

- Allows us to SELECT different values in a Specific Columns.
- Removes duplications
- Doesn't repeat the same value.

Student table.

Student_ID	Name	Subject	Marks
101	Rofhiwa	Maths	99
102	Mbali	Maths	20
103	Sipho	Physics	60
104	Tumi	Geo	85

SELECT DISTINCT Subject
FROM Student_table;

Subject
Maths
Physics
Geo

Data Filtering

WHERE Statement.

↳ Filtering Statement.
↳ it allows us to filter our data based on Specific conditions.
↳ It Affects the rows not the Columns.
↳ You filter rows based on a condition applied into a column.

SELECT Col1, Col2, Col3
FROM table_name
WHERE Condition

} WHERE Syntax.

Day 6. Data Filtering

Student-Id	Name	Subject	Marks
101	Rofhiwa	maths	99
102	Mbali	Maths	20
103	Siphe	Physics	60
104	Tumi	Geo	85

Give the list of students who got marks above 80

Give students doing maths

```
SELECT *
FROM Student table
WHERE Marks > 80;
```

Student Id	Name	Subject	Marks
101	Rofhiwa	Maths	99
104	Tumi	Geo	85

```
SELECT *
FROM Students-table
WHERE Subject='Maths';
```

Put Maths inside Single code because it is words

words or Date put in single code (gosh)

StudentID	Name	Subject	Marks
101	Rofhiwa	Maths	99
102	Mbali	Maths	20

AND Operator.

Give me list of students who are doing maths and got more than 50%.

```
SELECT *
FROM Students table
WHERE Subject='maths' AND Marks>50
```

* With the AND operator, both conditions must be satisfied.

or

```
SELECT  Name,
        Subject,
        Marks
FROM    Student_table
WHERE   Mark > 50 AND Subject = 'Maths';
```

Name	Subject	marks
Rofhiwa	Maths	99

→ Can start with any of the conditions

OR Operator

→ One of the conditions must be met.

Example

```
SELECT  *
FROM    Student_name
WHERE   marks > 50 OR Subject = 'maths';
```

Example 2: I want any student doing physics or have marks above 80

Student id	name	Subject	marks
101	Rofhiwa	Maths	99
103	Sipho	Physics	60
104	Tumi	Geo	85

```
SELECT *
FROM Student table
WHERE
marks > 50 OR
Subject = 'Maths';
```

IN Operator - Allows us to filter data by specifying multiple values in a column

Syntax

```
SELECT  Columns
FROM    Table_name
WHERE   Column IN ('Value1','Value2','Value3' ...
```

Example

Give me a list of students doing Physics, and Geo

```
SELECT  name,
        Subject
FROM    Students table,
WHERE   Subject IN ('Physics', 'Geo');
```

Name	Subject
Sipho	Physics
Tumi	Geo

Day 7.

Logical operator → To join multiple conditions (All or Both)

- you can put as many conditions as you like

AND Operator - Both conditions must be met

```
SELECT *
FROM table_name
WHERE Condition1 AND Condition2
      AND Condition3;
```

OR Operator

- One of the conditions must be met.

```
SELECT *
FROM table_name
WHERE Condition1 OR Condition2 OR Condition3;
```

IN Operator

- Allows us to filter data by specifying multiple values in a columns

```
SELECT * or Columns
FROM table_name
WHERE Column IN (value, value2, value3 ...);
```

BETWEEN OPERATOR

- It allows us to filter the data based on a specific range in our numeric or date columns.
- Range looks at the minimum and maximum
- Looks at the start point and the endpoint.

```
SELECT * or Columns
FROM table_name
WHERE Column BETWEEN min_val AND
      max_val
```

Write an SQL Script to give me names, & marks for students who got between 10% and 70%.

```
SELECT name,
       marks
FROM Student table
WHERE Marks BETWEEN 10 AND 70;
```

↓

Name	marks
Mbali	20
Sipho	60
Naledi	10

BETWEEN Statement is an inclusive statement it includes the 10 and 70.

NOT operator:

- It gives you the opposite result.
- It is a way of filtering data by saying what should be excluded.

```
SELECT  Columns
FROM    table_name
WHERE   NOT Condition;
```

* NOT BETWEEN, NOT IN

Student_ID	Name	Subject	Marks
101	Rofhiwa	Maths	99
102	Mbali	Maths	20
103	Sipho	Physics	60
104	Tumi	Geo	85
105	Naledi	Physics	10
106	Sabelo	Geo	71

Name	Marks
Rofhiwa	99
Tumi	85
Sabelo	71

```
SELECT name
       marks,
FROM Student_table
WHERE marks NOT
  BETWEEN 10 AND
  70
```

Day 8

Aggregate Functions

1. SUM() - Summarize the data
2. AVG() - Returns a single value.
3. MIN() - Consolidate the data
4. MAX()
5. COUNT()

- **An aggregate Function** - is a function that performs a calculation on multiple rows in a specific column and returns a single value or a single group.

Products

ID	Fruit	Revenue
102	Apple	4
101	Banana	2
103	orange	3
104	Kiwi	6

SELECT MAX(REV
FROM Products;

max rev
6

AS max

Syntax

SELECT MIN(REVENUE) AS min_rev
FROM products;

Return min_rev

Min_Rev
2

Product

ID	Fruit	Revenue
102	Apple	4
101	Banana	2
103	Orange	3
104	Kiwi	6

SELECT sum(REVENUE)
AS total_revenue.
FROM product;

Adding the values in Revenue Column.

total_revenue
15

4+2+3+6 = 15.

COUNT ()

- You Count Something that can be differentiated
- You Count Something that is unique.

Sum Money. add unique people.

Fruits..

order_Id	Product_Id	Product	Revenue
101	111	Banana	10
102	113	Apple	7
103	111	Banana	10
104	112	orange	5
105	111	Banana	10

* Tell me how many orders did we get.

SELECT COUNT (order_Id) AS num_orders
FROM fruits ;

- DO NOT Sum Id

num_orders
5

→ Can't Count ...

(1) COUNT() → How many rows
(2) SUM() → total of numeric columns.
↳ Addition of values.
(3) MIN() smallest values
(4) MAX() Largest value
5 AVG() Average values of numeric Column.

Sorting Statement.

ORDER BY { ASC - Ascending order (Small to big) 1-9, a-z
DESC - Descending order. big to small. 9-1, (z-a)

Products

ID	Fruits	Revenue
102	Apple	4
101	Banana	2
103	Orange	3
104	Kiwi	6

SELECT *
FROM Products.
ORDER BY revenue
DESC;

Results:

ID	Fruit	Revenue
104	Kiwi	6
102	Apple	4
103	Orange	3
101	Banana	2

SELECT
FROM Products
ORDER By revenue ASC

Revenue
2
3
4

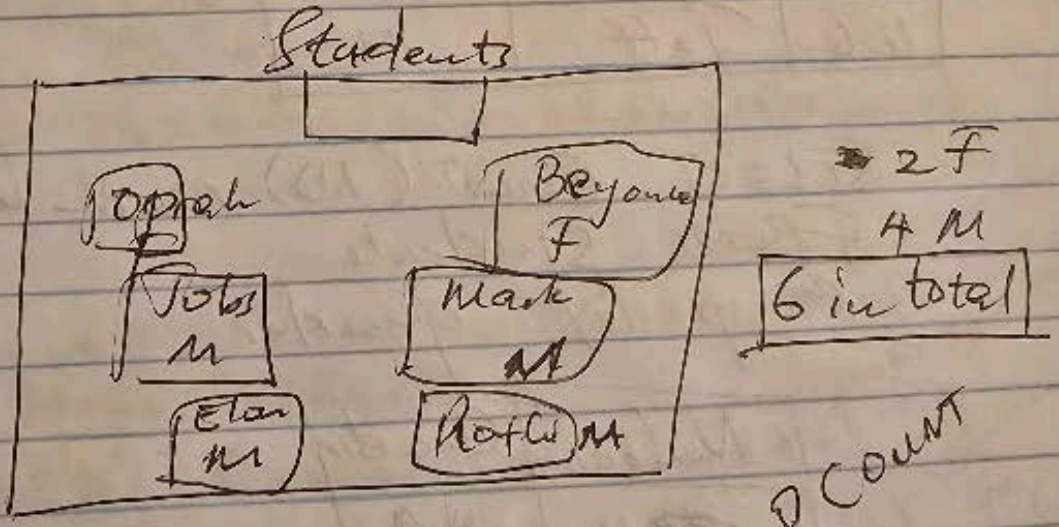
Day 4

- Used to summarize data
- Returns a single value
- It is a function that performs a calculation on multiple rows in a specific column and returns a single value or a single value per group.

Sum() - adds values
AVG() - find the average
COUNT() - Count rows
MAX() - highest values
MIN() - lowest values

In single terms :-

aggregate = Summarize a group of values into one result.



SELECT COUNT(*) AS CNT
FROM Students;

COUNT → Counts all the rows in your table.

If you want to know how many rows are in your table.

GROUP BY Statement

→ Group after aggregating.

→ Allows us to group our data by a specific category.

```
SELECT aggregate_function(Column) AS
       descriptive_name,
       Categorical_Column
FROM table_name
GROUP BY Categorical_Column;
```

Students

ID	Name	Gender
101	Rafli	M
102	Elon	M
103	oprah	F
104	Beyonce	F
105	Mark	M
106	Jeff	M

```
SELECT COUNT(ID) AS N_learners, Gender
FROM Students
GROUP BY gender.
```

N_learners	Gender
4	M
2	F

* We group by the column that is not aggregated in our Select Statement
* We don't mention the columns that is aggregate in our Group By clause

Day 10

Sorting Statement

* Does not remove or add anything.
- It allows us just rearrange.
* It allows us to rearrange our data without changing anything in that.

ORDER BY STATEMENT

Ascending order	Descending order
ASC	DESC
Smallest to biggest	Biggest to smallest
1 - 9	9 - 1
a - z	z - a

Syntax

```
SELECT  Column1  AS descriptive_name   → Rename the Column
        Column2,
        SUM(column3) AS descriptive_name,
        Column4
FROM    table_name
WHERE   condition
GROUP BY Categorical_Column
ORDER BY ASC;
```

WHERE → [illegible] condition

GROUP BY → When we have aggregated a column in our select

ORDER BY ASC → arrange the rows

Products_table

Product_id	Product_name	Revenue
111	Apple	2
112	Banana	4
113	Orange	3
114	Kiwi	5

```
SELECT Product_name,
       Revenue
FROM Products_table
ORDER BY Revenue ASC;
```

Product_name	Revenue
Apple	2
Orange	3
Banana	4
Kiwi	5

Small → Big

The descending order syntax

```
SELECT Product_name,
       Revenue
FROM Products_tabbe
ORDER BY Revenue DESC;
```

Results

Product_name	Revenue
Kiwi	5
Banana	4
Orange	3
Apple	2

Big → Small

LIMIT Statement

It allows you to specify the number of rows you want to see in your outcome.

Product_table

Product_Id	Product_name	Revenue
111	Apple	2
112	Banana	4
113	orange	3
114	Kiwi	5

```
SELECT *
FROM Products_table
LIMIT 2;
```

we normally use this to see the structure of the table

Product_Id	Productname	Revenue
111	Apple	2
112	Banana	4

Data Inspection

Day 11

HAVING Statement

The HAVING Statement allows us to filter data that has already been aggregated

Syntax

```
SELECT Column1,
       agg_fun(column2) AS description_name
FROM   table_name
GROUP BY Column1
HAVING Condition;
```

HAVING Statement allows to filter data that is grouped

Employees

Id	Name	department	Salary
101	mark	HR	10
102	Steve	HR	20
103	Jeff	IT	30
104	Elon	IT	15
105	Rofhi	CEO	100

(1) SUM()
(2) AVG()
(3) COUNT()
(4) MIN()
(5) MAX()

Write an SQL query to calculate the total Salary paid per department

Syntax

```
SELECT department,
       Sum(salary) AS total_salary
FROM Employees
GROUP BY Department;
```

→ Categorical Column

- You group by a column that is not aggregated in your SELECT statement (categorical column).

Out Come

department	total_salary
HR	30
IT	45
CEO	100

Grouped data

Id	Name	department	Salary
101	Mark	HR	10
102	Steve	HR	20
103	Jeff	IT	30
104	Elon	IT	15
105	ROFL	CEO	100

+ Write a SQL query to show me the department that get salary allocation of more than 40

Syntax

```
SELECT department,
       Sum(Salary) As total_Salary
FROM EMPlayees
GROUP BY department
HAVING total_Salary > 40 ;
```

department	total_Salary
IT	45
CEO	100

WHERE Statement - filters raw data
HAVING Statement - filters grouped data or aggregated data.

*For you to use a HAVING Statement you need to first aggregate your data and group it to Specific Categories then you can filter the grouped data using the HAVING statement.

Employee_id	Name	department	Salary	Bonus
1	Tom	IT	60 000	5000
2	Jerry	HR	55000	4000
3	Spike	FINance	70 000	6000
4	Tyke	IT	62000	5500
5	Butch	HR	54000	3500

```
SELECT department,
       Sum Salary AND Bonus as total_
       Salary and total_bonus
FROM   Employee
```

GROUP BY department;

depart	total_Salary	total_bonus
IT	122 000	10500
HR	109000	7500
Finance	70 000	5000

```
SELECT department,
    Sum Salary as total_Salary
    Sum bonus as total_bonus
FROM Employee
Group By department;
```

(1) [illegible]

Business Analytics

- Data analytics
- Data science
- Data engineering

* Transforming raw data into actionable insights
* Analyze business performance
  - Product performance.
* We want to drive business decision using data to better plan for the future.
* Drive Business Growth → Business Profitability

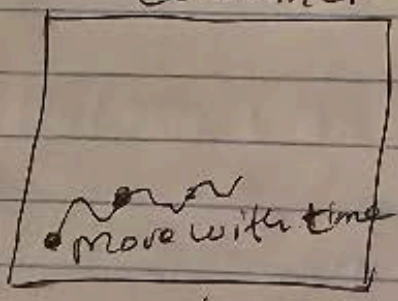
- Giving solutions to business problems

① Understand the business
- Domain Expertise
- Business model
  - Understand how business makes money

② Identify the problem
- Data analytics
- Driven by data

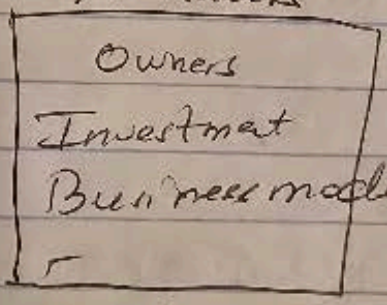
③ How do we measure success (KPI)

Consumer



- What Consumers are Saying? (2)

Producers



- What is required to run the business (1)

Launch Product

Qualitative KPIs

- Bring in new customers (market share)
- Sell more, Diversify, Increase revenue
- Increase ~~revenue or profits~~, Reduce cost
- Remain competitive, Brand visibility, Solve a problem
  - ↳ Positive Reviews
- Customer Loyalty or more investment
  - ↳ Repeat customers
  - ↳ Tenure

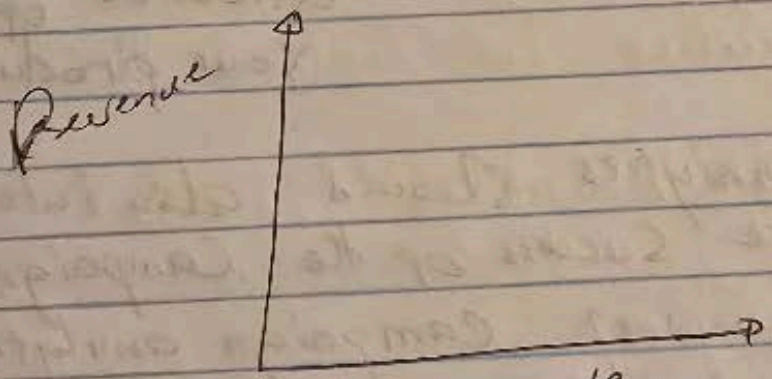
Quantitative KPI

Stock price

Sentiment analysis.

Quantitative KPI

Revenue = Price x Quantity ($R = P \times q$



Goal of Business analytics is to reduce cost

~~Cost~~

Total Cost
- variable
- Fixed

Total cost = Variable cost + Fixed
- ↳ linked to products we sell in the business

Number of transaction → 5 ↑ frequency.

Number of units Sold → 1000 ↑

Number of Customers → 10 ↑

Profit = Revenue ↑ − Cost ↓ P = R − C

Campaign - Stimulate Customers in the direction of Purchasing your product.

- Awareness
- Attract Customers
- Customer acquisition
- Introduce new product
- Reduce stock holding.

Business Analytics should kind of inform what campaign should we do.

↳ To act in the direction of purchasing your product

* Business Analytics should also inform how we measure the success of the campaign.

Tell us what to do & Stimulate

↳ Post campaign analytics

Business Analytics

KPI	target		Control group	
	Pre	Post	Pre	Post
Revenue	10K	25K	5K	6K
Frequency				
Average Revenue per customer	100	220	80	85
Conversion rate				

→ Don't tell anything to do

→ No Stimulation

Average Revenue per User

Dormant / Churn - Inactive
Active

↳ Defined by active days.
↳ When last did you transact with us

AI Day.

AI tools

Gemini	Kimi	Zapier
Chat GPT	Base 44	
Claude	Replit	Whisper flow
meta AI	Manus	
Grok	Conses	Smart cat
Deep seek	loveable	Julius AI
Copilot	Versel	Quillbot
Notebook LM	Cursor	Genie AI
Gemma	Lelapa AI	
Canva	Kiro	
Perplexity	DELL A	

GPT - Generative Pre Trained Transformer

LLM - Large Language Model

Machine learning -

AI - Artificial Intelligence :

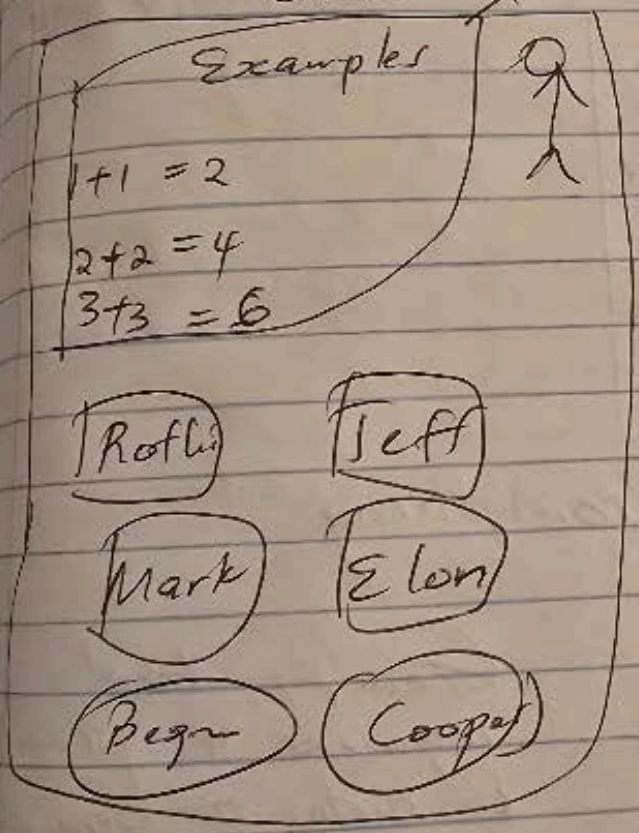
↳ For an AI system to exist it needs to learn patterns from data

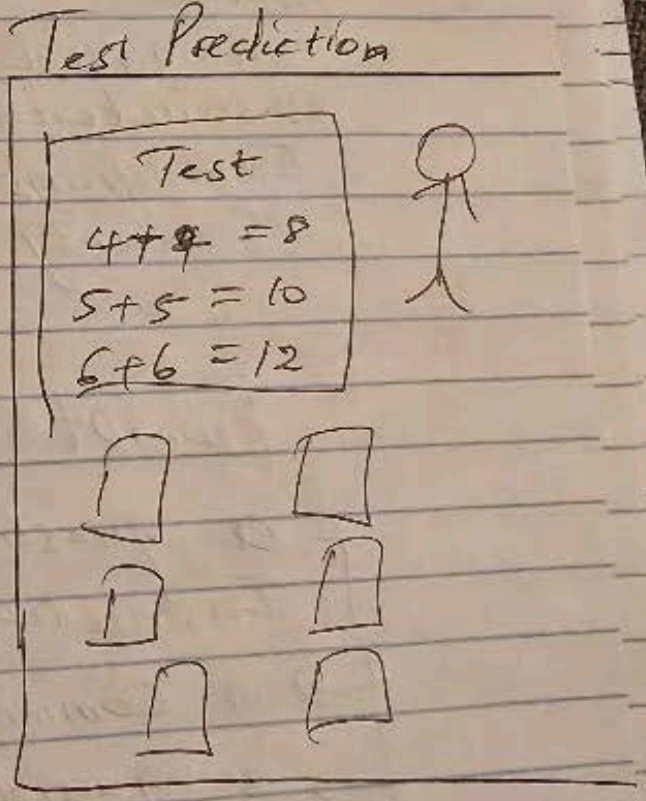
↳ Training.

Machine Learning - Enables a machine to learn patterns
- learns patterns from data

↳ Branch of Artificial Intelligence (AI) where computers learn patterns from data systems.
- To make decisions.

Class → learning patterns from data





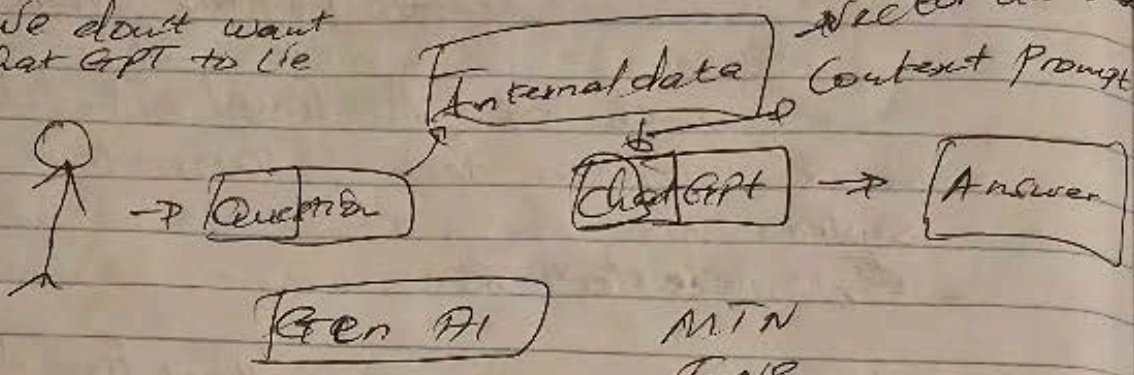
Hallucinating - makes things up that are incorrect

↓

Solution to hallucination

RAG - Retrieval Augmented Generation → minimize hallucination → Chatbot

↳ We don't want Chat GPT to lie



Gen AI

MTN
FNB
Standard Bank

Tokens
Tokenization
Unstructured data
Embeddings
Similarity Search

Prompting.

Prompt
- a question
- Instruction
- a command
- a guardrail
- a request

Gen AI tool requires a prompt in order to generate what you need

Prompt engineering → clear instruction
→ Detailed instruction
→ Specific instruction

↳ A way to construct an effective prompt to ensure clarity to the AI so that it does not give you the wrong answer

Different ways to Prompting

① Few-Shot Prompting
- You give the LLM few examples so that it understands your question
- Is a way of interacting with a LLM (Gen AI) by asking it question and give it examples as part of the Context for it to be able to understand what you want.

② Zero Shot Prompting.
- Direct
- this is when you ask a question without giving any example.

③ Contexual Prompting.
- you give background information

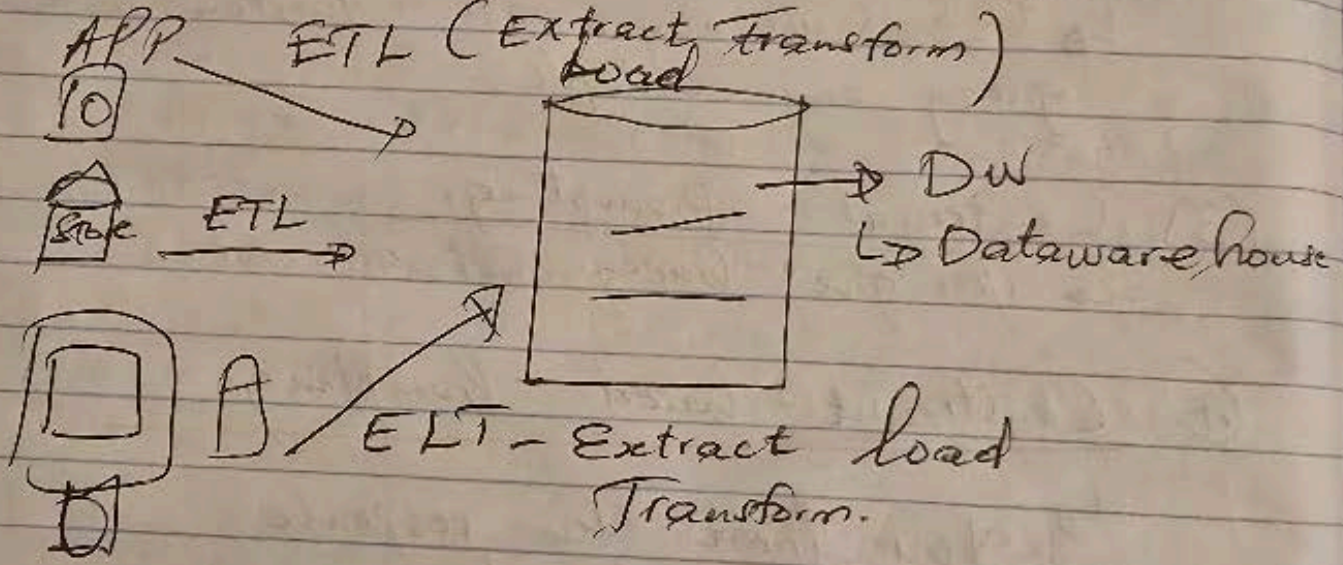
④. Constraint - based Prompting.
- you limit the response.
- Restriction based
  - Controlling the kind of output you want from the LLM
  - -> Limiting the scope of the output.

Machine Learning Process

* The outcome of a machine learning process is an AI tool / system.
* In order to build a machine learning model we need data.

* The role of a Data Engineer is to make the data available in the datawarehouse.

Build data pipelines to store the data in the warehouse.



Functions

1. Data Collection
2. Stakeholder: Define the problem

- Before building AI/ML solution; we need to define the problem we are trying to solve.

Bank - Predict the likelihood of a customer leaving my bank and going to another.

Goal - we want to retain our customers.

FNB ✕ Capitec

- I want to create a machine learning model to predict the customers likely to churn.

(3) Explore and understand data.
↳ Exploring Data Analysis (EDA)

Correlation

* You analyse the data to understand patterns, trends and relationships.

Asking data driven Questions.

↳ which customers are leaving
↳ Does age effect Churn.
↳ Do customers with many complaints leave more.

4 Prepare the data for machine learning.

↳ Feature Build
- Converting text into numbers (Encodi
- Scaling numeric values e
- Creating new useful columns
- Splitting data into feature and target

Feature engineering.
Feature Selection

(5) Split the data (Train-Test Split)

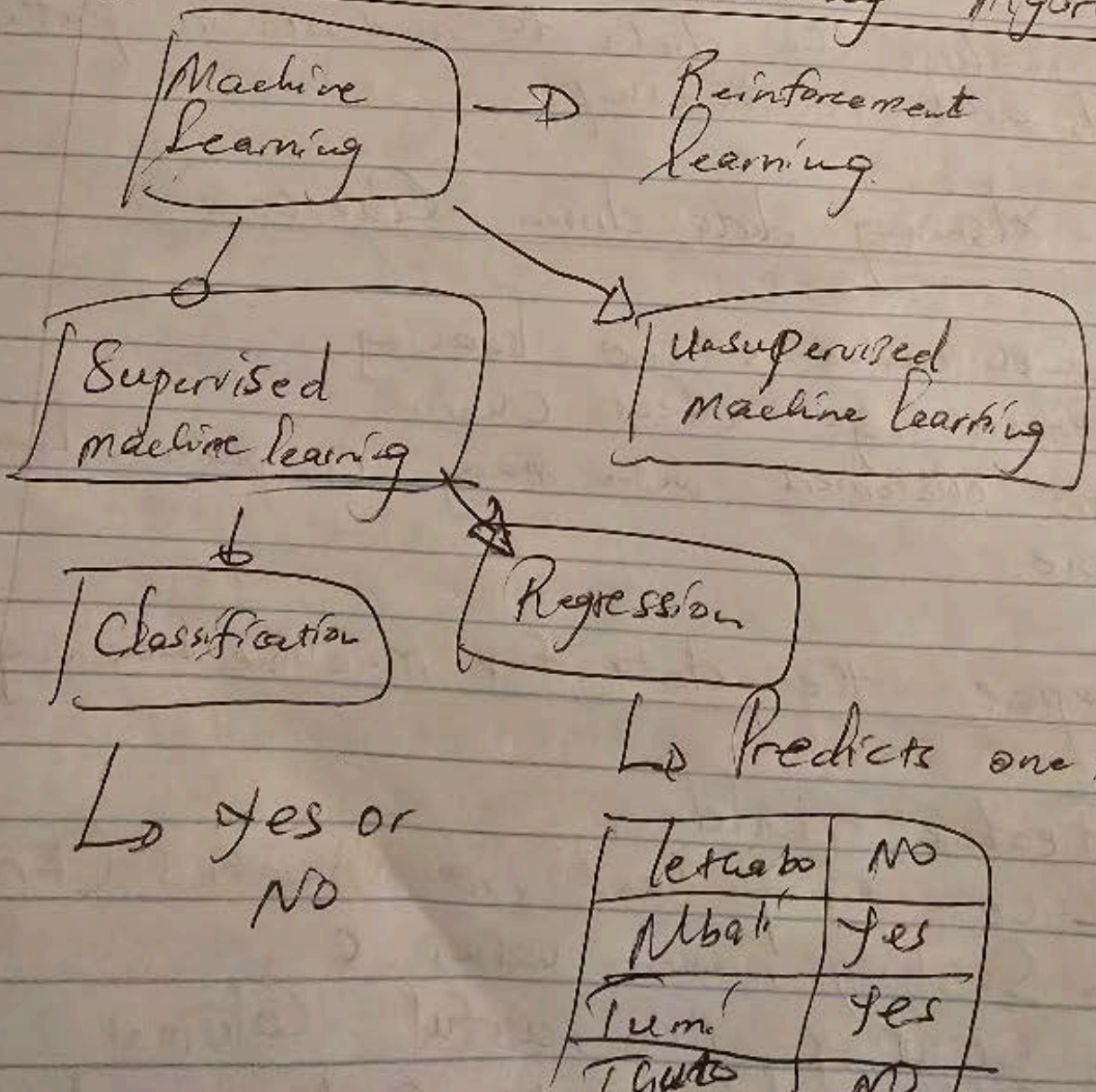
Training data - used to teach the model (80%)

Testing Data - used to check if the model works (20%).

1	5
2	6
	7
3	
4	8

9
10

(6). Choose Machine learning Algorithm.



Ketuabo	NO
Mbali	yes
Tumi	yes
Thando	NO

Case Statements

- A Case Statement allows us to build logical conditions.

```
SELECT    Column1,
          CASE
               WHEN Condition 1 THEN 'Result1'
               WHEN Condition 2 THEN 'Result2'
          ELSE 'default result'
          END AS Column_name,
          Column2,
          Column3
FROM table_name
```

Use built conditions based on other Columns

↳ A Case Statement creates a new Column.

Default result only comes out when none of the conditions are met.

Example

Products

productid	Product name	price	price_Category
1	Laptop	12000	Expensive
2	Phone	800	Midrange
3	Keyboard	45	Budget
4	Monitor	300	Midrange
5	Mouse	25	Budget

↑ Results

Syntax

```
SELECT productname,
       price,
       CASE
           WHEN price > 1000 THEN 'Exercise'
           WHEN price BETWEEN 100 AND 1000
               THEN 'midrange'
           WHEN Price < 100 THEN 'Budget'
       END AS price_category
FROM Products;
```

amount ≥ 500 AND amount ≤ 999.99

orders

order_id	customer_name	amount
1	Alice	150
2	Bob	560
3	Charlie	999.99
4	Diana	45.50
5	Ethan	1200
6	Rafhiwa	x

Results

order_value	category
low-value	
medium value	
medium value	
low value	
High value	
Null	

label for each

* High value for orders > 1000
* Medium Value for 500 - 999.99
* Low Value for orders < 500

```
SELECT
        Customer_name,
        amount,
    CASE
        WHEN amount >1000 THEN 'High value'
        WHEN amount BETWEEN 500 AND 999.99 THEN
                                'medium value'
        WHEN amount < 500 THEN 'Low Value'
    END AS order-value_category
FROM orders;
```

```
SELECT
    CASE
        WHEN amount >, 1000 THEN 'High value'
        WHEN amount BETWEEN 500 AND 999.99 THEN
                                'medium Value'
        WHEN amount <500 THEN 'Low value'
    END AS order_Value_Category;
~~FROM orders;~~ Customer name
                 amount
FROM orders;
```

Continued CASE Statement

```
SELECT
    Column1,
    Column2,
CASE
    WHEN Condition1 THEN 'Result1'
    WHEN Condition2 THEN 'Result2'
    WHEN Condition3 THEN 'Result3'
END AS Column_name,
    Column 3,
    Column 4
FROM table_name
```

- We build the conditions based on other columns
- A case statement creates a new column.
- A Case statement is normally placed within the SELECT statement
-

Question & Example

Classes

Classid	Subject	enrolled_Stud	Class_Size_Category
1	Math	30	large
2	English	25	large
3	Science	15	medium
4	Art	5	small
5	History	20	medium

conditions

* >= 25 => large
* 10 - 24 => Medium
* < 10 => small

Syntax

```
SELECT Subject,
       enrolled_Students

  CASE
    WHEN enrolled_Students >= 25 THEN 'Large'
    WHEN enrolled_Students BETWEEN 10 AND 24 THEN 'medium'
    WHEN enrolled_Students < 10 THEN 'small'
  END AS Class_size_Category
FROM classes;
```

Second Example

Product_Inventory

Product_id	Stock_qty	Stock_Status
1	5	low Stock
2	0	out of Stock
3	25	In Stock
4	10	In Stock
5	3	low Stock

Conditions

0 → Out of Stock

1–5 → low Stock

5 → In Stock

BETWEEN Statement
eg 1 and 5
→ Excludes 1 and 5

Syntax

```
SELECT Product_id,
       Stock_qty,
   CASE
      WHEN Stock_qyt = 0 THEN 'out of Stock'
      WHEN Stock_qyt BETWEEN 1 AND 5 THEN 'low St
      WHEN Stock_qyt > 5 THEN 'In Stock'
   END AS Stock_Status
FROM Product_Inventory;
```

>= 5 = include 5
> 5 does not

attendance.

Student_id	days_present	total_days	attendance percentage	attendance Status
1	45	50	90%	Excellent
2	30	50	60%	Need improvement
3	48	50	96%	Excellent
4	25	50	50%	Need improvement
5	50	50	100%	Excellent

Conditions
1) 90% → Excellent
75% - 89% → Good
< 75% → Need improvement

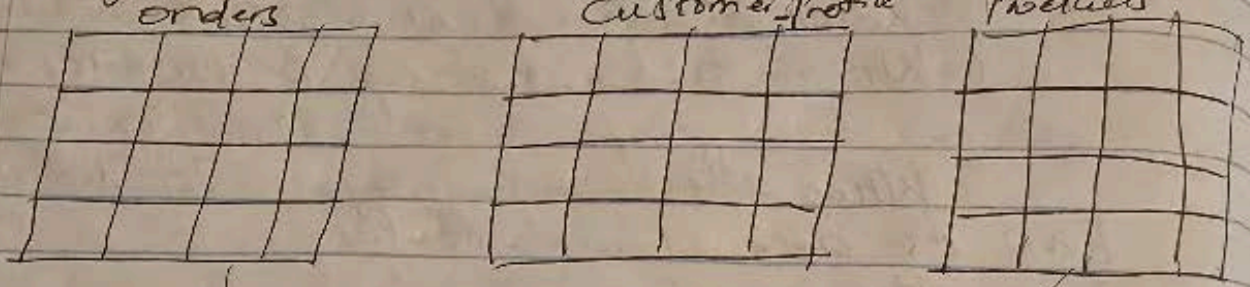
```
SELECT Student_id,
       (day present/total days)*100 AS attendance_
                                       Percentage,

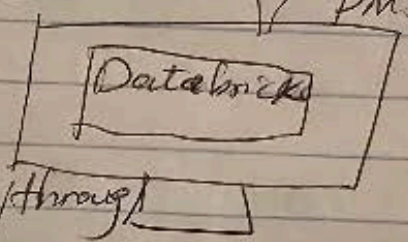
    CASE
        WHEN attendance_Percentage >=90% THEN 'Excellent'
        WHEN attendance_Percentage BETWEEN 75% AND
                                   89% THEN 'Good'
        WHEN attendance_Percentage <75% THEN 'Needs
                                   improvement'
    END AS attendance_Status
FROM attendance;
```

Joins and Unions

Data Analysis Process

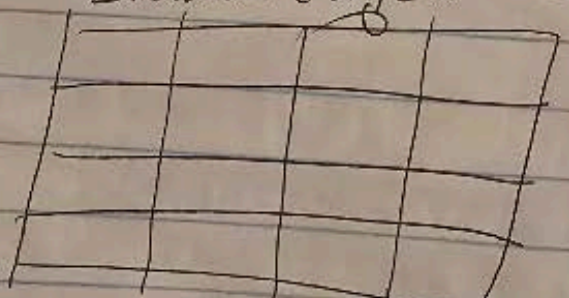
*Big data - Stored in a Datawarehouse

① 

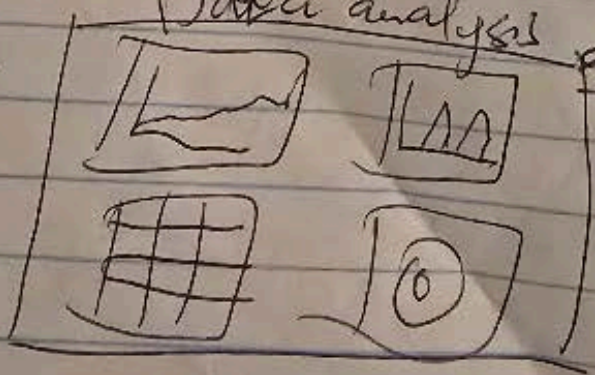
② 

PMS Database Management System

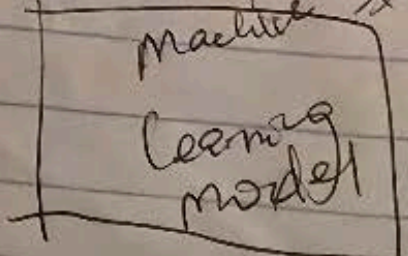
coding through SQL Pyspark → Aggregating data - Data processing
- Cleaning data
- Joining the tables

③ 

- Summarized Data
- Cleaned data
- Aggregated data.

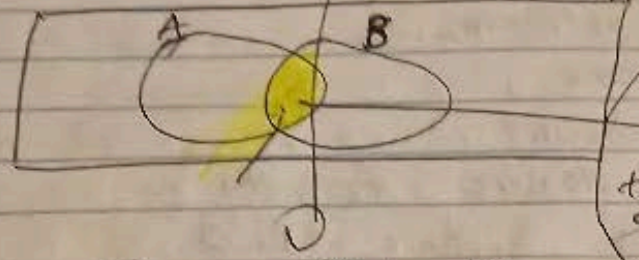
④ 

→ PowerBI



→ Data Scie

Inner Join → You join two or more tables based on a common Column



* Returns only the rows where there is a match between the table joined together on the common Column

* Joins effect the rows

- You join table on the common Column but the effect is on the rows.
- When you join two or more tables, you can select any common between the tables

Example.

Syntax.

```
SELECT Column1,
       Column2,
       Column3
FROM table1 AS A
INNER JOIN table2 AS B
ON A.Common_Column = B.Common_Columnj
```

Customers

Customer_id	name
1	Rofhi
2	Mbali

Orders

order_id	Customer_id	amou
101	1	300
102	4	150
103	5	100

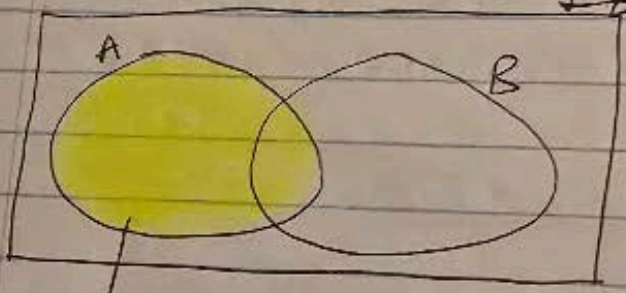
Syntax

```
SELECT customer_id
       name ,
       amount
FROM   Customers AS A
INNER JOIN orders AS B
ON A.Customer_id = B.Customer_id.
```

Customer_id	Name	amount
1	Rofhi	300

* No Match for ~~MGabi~~.

~~LEFT~~ JOIN



↳ We call the left table ~~our~~ base table
↳ We return every row in the left table and add the matches from the right table.
↳ If No match, we put NULL

Syntax

```
SELECT Column1,
       Column2,
       Column3
FROM table AS A
```

LEFT JOIN table2 AS B
ON A.CommonColumn = B.Common_Column;

Customers

Customer_id	Name
1	Rafhi
2	Mbali

orders

order_id	Customer_id	amount
101	1	300
102	4	150
103	5	120

```
SELECT Customer_id,
       name,
       amount
FROM customers
LEFT JOIN orders
ON Customers.Customer_id = orders.Customer_id
```

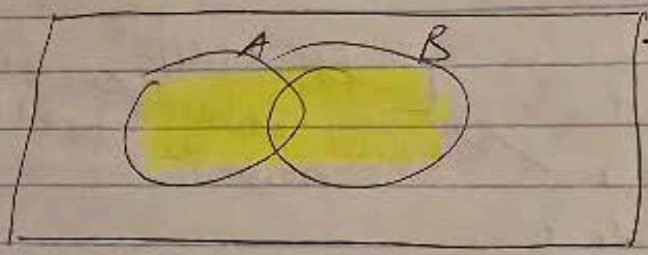
→ Left table
Have return every row from the left table

Customer_id	name	amount
1	Rafhi	300
2	Mbali	NULL

JOINS

- Joins tables based on a common column.
- Joins affect the rows
- INNER JOINS returns rows that are mat
- When you join two or more tables, you ca
Select any columns between the two tables.

FULL OUTER JOIN



→ Community of pro

→ Returns everything

- Returns all the rows between two table an match/combine the rows where there is a match.

Syntax.

```
SELECT column1,
       Column2,
       Column3
FROM table1 AS A
FULL OUTER JOIN table2 AS B
ON A.Column = B.Column;
```

Customer_id	Name
1	Rath
2	Mbali

Order_id	Customer_id	amount
101	1	300
102	4	150
103	5	100

Syntax

```
SELECT Customer_id,
       name,
       order_id,
       amount
FROM customers AS A
FULL OUTER JOIN orders AS B
ON A.customer_id = B.customer_id;
```

customer_id	name	order_id	amount
1	Rofhi	101	300
2	Mbdi	NULL	NULL
4	NULL	102	150
5	NULL	103	600

another Syntax

```
SELECT customer_id,
       name,
       ~~amount~~ order_id,
       amount.
FROM customers
FULL OUTER JOIN orders
ON Customers.Customer_id = orders.Customer_id;
```

```
SELECT   A.Customer_id,
         A.name,
         B.order_id,
         B.amount
FROM Customers AS A
FULL OUTER JOIN orders AS B
ON A.Customer_id = B.Customer_id;
```

UNION AND UNION ALL

- This is a way to combine two tables that are similar

- Have the same number of columns

Online_Customers

Customer_id	name
101	John
102	Mary
103	Peter

Store Customers

Customer_id	name
102	Mary
103	Peter
104	Sarah

* Give me a list of all unique Customers that Shop with us.

Syntax.

```
SELECT cust_id,
       name
FROM   online_customers
UNION
SELECT Cust_id,
       name
FROM   Store_Customers;
```

Union
Removes duplicates

Cust_id	name
101	John
102	Mary
103	Peter
104	Sarah

Union Removed duplicates.

*Mary and Peter appeared in both tables but in the outcome table, they appear onc

UNION ALL keeps everything and keep duplicates.
↳ Combines results and keeps all duplicates

Syntax

```
SELECT  Cust_id,
        name

FROM    Online_Customers
UNION ALL
  SELECT    Cust_id,
            name
  FROM      Store_Customers;
```

Cust_id	name
101	John
102	Mary
103	Peter
102	Mary
103	Peter
104	Sareh

~~Main difference~~

~~Feature~~



Main difference

Feature	UNION	UNION ALL
Remove duplicates	Yes	NO
Keep all rows	NO	Yes
Faster	Slower	Faster
Best used when	You want unique records	When you want all records

* Important rule: The SELECT statements must have all the same number of columns and columns must have compatible data types.

January_Sales

product_id	Product_name
P001	Laptop
P002	Mouse
P003	keyboard

February_Sales

Product_id	Productname
P002	mouse
P003	keyboard
P004	Monitor

UNION Syntax

```
SELECT  Product_id,
        Product_name
FROM    January_Sales
UNION
SELECT  Product_id,
        Product_name
FROM    February_Sales;
```

Product_id	Product_name
P001	Laptop
P002	Mouse
P03	Keyboard
P004	Monitor

Union All Syntax

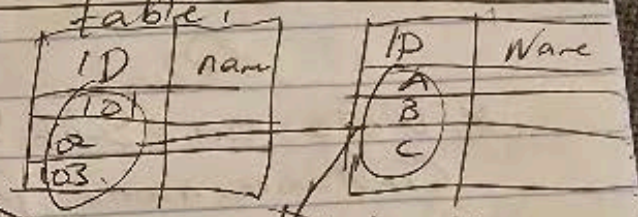
```
SELECT  Product_id,
        Product_name
FROM    January_Sales
UNION ALL
SELECT  Product_id,
        Product_name
FROM    February_Sales;
```

Product_id	Product name
P001	Laptop
P002	Mouse
P03	Keyboard
P002	Mouse
P003	Keyboard
P004	Monitor

UNION and UNION ALL Summary.

UNION	UNION ALL
- Remove duplicates - Needs same column names - Need same data type with the Column - Columns must be arranged in the same order in both statement	- Does not remove duplicates - Needs same same column names - Needs same data type with the column - Columns must be arranged in the same order in both statements

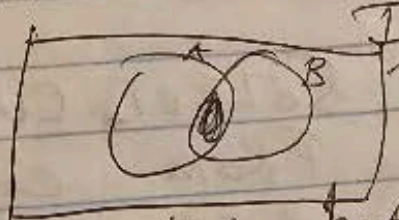
```
SELECT Column1,
       Column2,
       Column3
FROM Table1
UNION
SELECT Column1,
       Column2,
       Column3
FROM Table2;
```



Match the position in your Select statement

INNER JOIN

* We need a common column between the tables we are joining.
* Joins affect the rows.
* Returns a match
* Rows that do not match between two tables are excluded from the outcome
* A INNER JOIN is used to return only record that have matching values in both tables.



Customers

Cust_id	Customer_name
101	John
102	Mary
103	Peter
104	Sarah

orderid	Cust_id	Produ
1	101	Lapto
2	102	mouse
3	103	keybo

* When you join two or more tables, you ca Select any table from the table you are joining

Q1

* Write an SQL script to give me the Cust. Customer_name, Product using INNER JOIN

Syntax

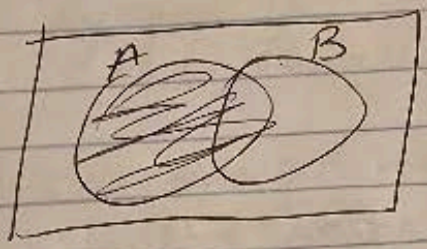
```
SELECT   Cust_id,
         Customer_name,
         Product
FROM     Customers
INNER JOIN orders
ON Customers. Customer_id = orders.
   Customer_id ;
```

or

```
SELECT Cust_id, Customer_name, Product
FROM Customers AS A
INNER JOIN orders AS B
ON A. Cust_id = ON B. Cust_id
```

Cust_id	Customer_name	Product
101	John	Laptop
102	Mary	Mouse

LEFT JOIN



Returns everything on the LEFT Table.

Syntax.

Main table
Left table
Base table

```
SELECT  Column1,
        Column2,
        Column3
FROM    table1 AS A
LEFT JOIN table2 AS B
ON A.Common_Column = B.Common_Column;
```

Right table

```
SELECT  cust_id,
        Customer_name,
        Product
FROM    Customers AS A
LEFT JOIN Orders AS B
ON A.cust_id = B.cust_id.
```

Cust_id	Customer_name	Product
101	John	Laptop
102	Mary	mouse
103	Peter	NULL
104	Sarah	NULL

Date Functions.

Date
↳ A data type that shows periods of the data collection

2026 - 05 - 28

DAYNAME (2026-05-28) - Extract the day name of the week based of your date/column
↳ Thursday.

MONTHNAME (2026-05-28) - Extract the month name based of your date column.
↳ May

YEAR (2026-05-28) - Extract the year value based on your date column.
↳ 2026

DAY (2026-05-28) - Extract the day value from your date column.
↳ 28

DataBricks.

Using retail Sales.

```
SELECT * Customer_id, Date
FROM retail sales
LIMIT 10 ;
```

```
SELECT Customer_id,
       Date,
       YEAR(Date) AS trans_year, -Extract ye
       MONTH(Date) AS trans_month, -Extract m
       DAY(Date) AS trans_day -Extract da

FROM retail_sales
LIMIT 5 ;
```

* MONTHNAME(Date) AS trans_month_name,
  DAYNAME(Date) AS trans_day_name
  *Extract the day name from the date column
  Extract the month name from date Column

```
SELECT COUNT(DISTINCT customer_id), AS Unique_Customers
-- Comment - Counting different Customers in a table
       Date
YEAR(Date) AS trans_year, Extract year from Colum
MONTH(Date) AS trans_month [redacted] Extract month [redacted]
Day(Date) AS
MONTHNAME(Day) AS trans_month_name -- Extract
 month name from date column.
DAY(Date) AS trans_day -- Extract the day
DAYNAME(Date) AS trans_day_name, Extract
name
Sum('Total Amount') As [redacted] revenue - Calc total
FROM retail_sales
```

GROUP BY ALL — grouping my data based on all the columns that are not aggregated in my select statements

LIMIT 05;

Date Functions Part 2. Current_Date.

```
SELECT Current_date() As today_dt;
    -- Gives us todays date

SELECT Current_date()-1 As Yesterday_dt;
-- this function gives me yesterday's date
   because I put minus 1

SELECT Current_timestamp() As datetime;
-- this function is used to get the current
   date and time.

-- TO_DATE (String and format) - Converts
   a string to (text/words) to a date.
      try
SELECT to_date('2026-06-03', 'YYYY-MM-DD')
   As dt;

-- TO CHAR(), Converts a date into a
String.          date_format()

SELECT date_Format (Current_date(), 'YYYYMM')
As Month_id

SELECT Current_date() As today_dt,
       date_format (Current_date(), 'YYYYMM'
          As Month_id;
```

~~-- DATED_FN()~~

Converting to your time zone.

SELECT Convert_timezone ('UTC', 'CUST +2',
Current_timestamp()) AS datetime;

DATED IFF() - We use this function to calc the difference between two dates.
~~SELECT datediff~~

DATED DIFF (end_dt, start_dt) Date.

SELECT datediff (Current_date(), Current_d
() -3);

-- Date_Sub - It Subtract the number of days fro date colum
SELECT date_Sub (Current_date(), 2) As
trans_dt

-- Date_add (date, number of days) - It add
~~SELECT date_add~~ the number of days
from the date column.

SELECT date_add (to_date ('2026-01-01'), 5
As break_up_dt

NULL Functions

NULL - is an empty space.
- is a missing value.
- is a blank space.
- is no value.

Null-function - help us to deal with null value in our data.

- How do we know that we have a Null value in our data.

* IS NULL - this is a function that help us to find a row of a specific column that it has a null value.

Syntax: IS NULL

```
SELECT Column1,
       Column2,
       Column3
FROM table_name
WHERE Column IS NULL;
```

The WHERE Statement filter the rows based on a specific column.

Customers

Cust_id	Name	email
1	John	John@gmail.com
2	Mary	null
3	Peter	Peter@gmail.com

```
SELECT *
FROM Customers
WHERE email IS NULL;
```

Cust_id	name	email
2	Mary	NULL

2. IS NOT NULL

- this is the opposite of IS NULL
- there is a value and not an empty space.
- this is a way of finding rows where a specific column is not null or empty

Syntax

```
SELECT *
FROM Customers
WHERE email IS NOT NULL;
```

Cust_id	name	email
1	John	john@gmail.com
3	peter	peter@gmail.com

How to replace a NULL value

IF NULL (expression, default_value)
↳ is a way to replace NULL values with an alternative value of your choice.

Customers

Cust_id	Name	email
1	John	john@gmail.com
2	mary	null
3	Peter	Peter@gmail.com

```
SELECT cust_id,
       name,
       IFNULL(email, 'No email') AS updated_email
FROM Customers;
```

Cust_id	name	updated_email
1	John	John@gmail.com
2	mary	No email
3	Peter	Peter@gmail.com

COALESCE ( ) IFNULL ( )

Replaces Null with another value.

→ COALESCE(expr1, Expr2, expr3, ... exprn)
↳ can check multiple columns and return the first non-null value:

Customers

Cust-id	name	Primary_Phone	Secondary_phone
1	John	071 232 4354	Null
2	Mary	Null	078 7372 893
3	Peter	null	null
~~4~~	~~Sarah~~		

Syntax

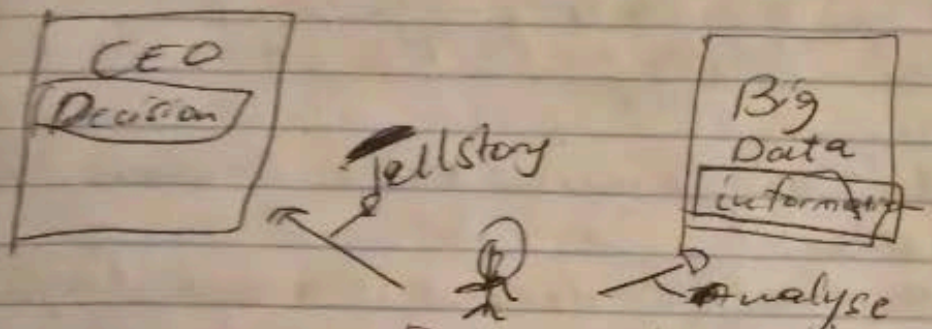
```
SELECT cust-id,
       name,
       COALESCE(primary_phone, secondary_Phone,
       0000) AS Contact
FROM Customers;
```

Cust-id	name	Contact
1	John	071 232 4354
2	Mary	078 737 2893
3	Peter	0000

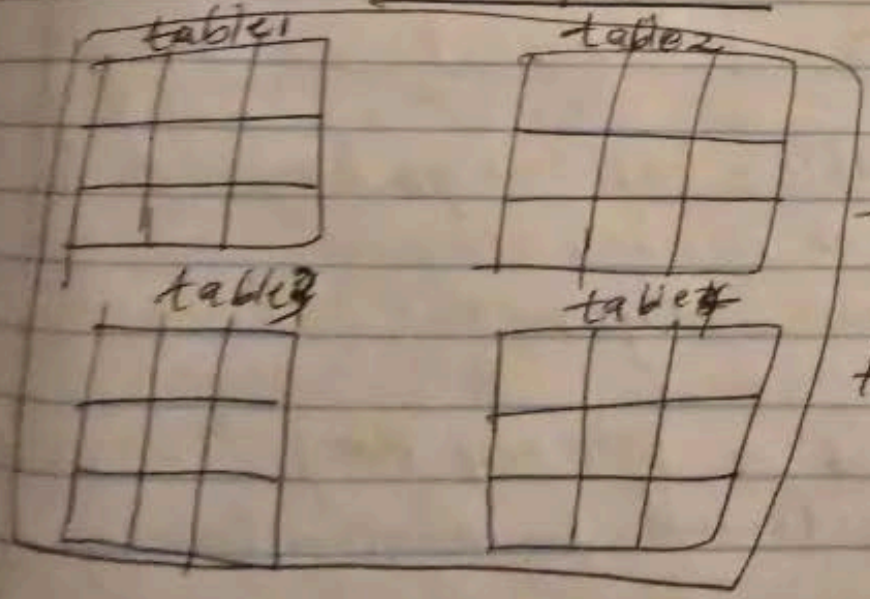
Data Visualization

Data - is information.

* You need to be able to tell a story from your data
* You need to drive/extract actionable insights from your data.
  ↳ Drive business decisions



Data Process

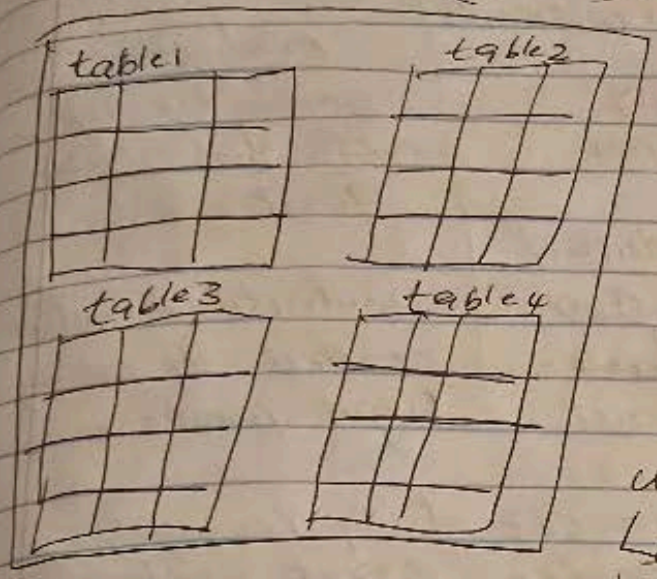


Relational database
↳ stores structured data

~~Data warehouse~~
that can be related & joined together to become.

* In a relational database we store structured data (dimension tables or fact tables) that can be connected to one another using primary key and foreign key.

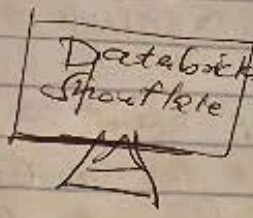
Datawarehouse (OLAP)



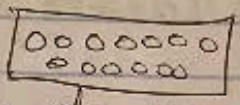
access the data

connected data to our

Data Management System DMS



In a DMS we code.

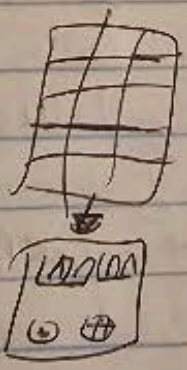


Why do we need to code

- Data Inspection to see what data is available.
- Data exploratory Analysis
  - To understand the relationship between our data points.
- Feature building
  - creating features used in ML for Predictions.

we code to retrieve, to amend, to update, to manipulate, to clean to aggregate, to join, to delete with the goal of Extracting valuable/actionable insight.

- Summerized table
- Aggregated table
- Combined table
- Unified table.
- Insight table



BRS (Business Requirements

THE Ask - THE WHY
THE REASON BEHIND

CEO: I want to understand the revenue for our Coffee Shop for different products.

Analyst
Answer the ask -
Ask data driven questions

understand what is needed by asking the right questions.

* Would you like to see daily revenue?
* Would you like to see repeat customer?
  - To analyse the frequency.
  -
* Do you want to see revenue with VAT included or excluded?

Data Processing - Code
↓
Data Analysis - Insight Generation
↓
Data Visualization - Creating Graphs
↓
Data presentation - Putting them together

Day 2 Data Visualization

* CEO - I want to understand the revenue for our coffee shop for different products.
* The CEO wants to track the daily revenue from Coffee Shop

What KPI's would like to measure?
Revenue? Number of Customers?
Distinct Products?

Y-O-Y
M-O-M
W-oW - week over week
MTD - Month to date
YTD - Year to date
T-1 vs T-7

The Data Design Blueprint.

- Choosing the right chart for effective story telling

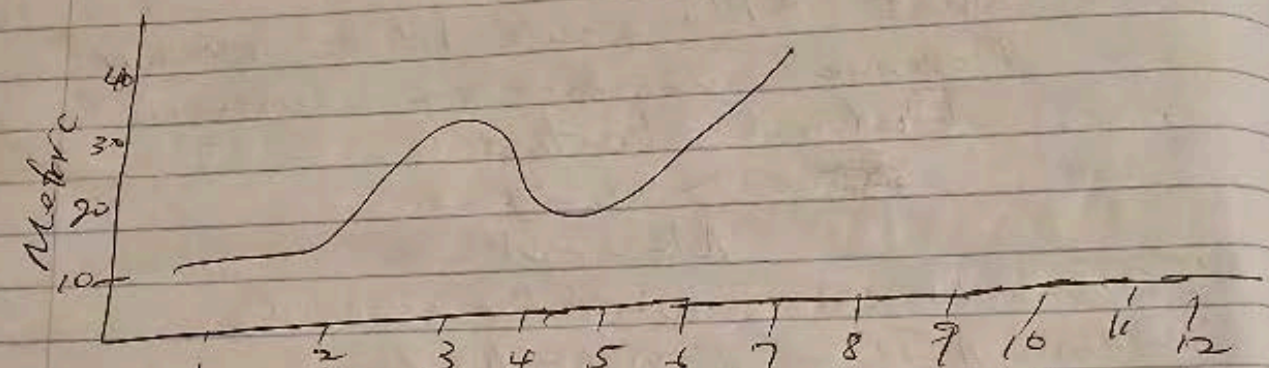
Bar Chart
Metric
Years
legend - Split
male
female
Clothing Beauty Electronics
Categorical data

When to use?
- Comparing categories.
- categorical data is grouping data
- is data that puts our information into groups mainly - words eg gender

Metric - refer to something we are measuring
- Examples - Revenue
- Number of sales/transactions.
- Number of Customers.
- ~~Numeri~~ Numerical or quantitative data

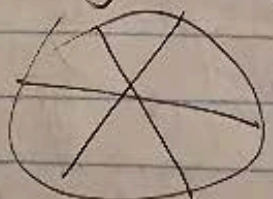
*A split / legend gives you detailed breakdown & granularize

Line chart → showing trends over time

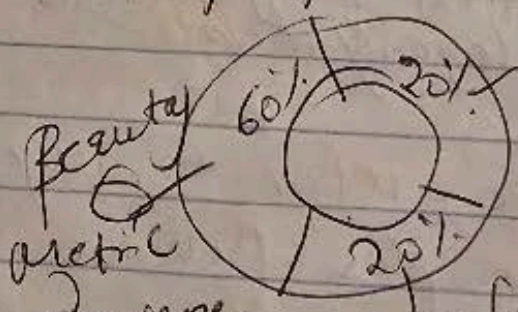


Date / Time → Continuous trends
- Month
- Day of week
- Years - 2022, 2023, 2024, 2025, 2026
- Date
- Hours

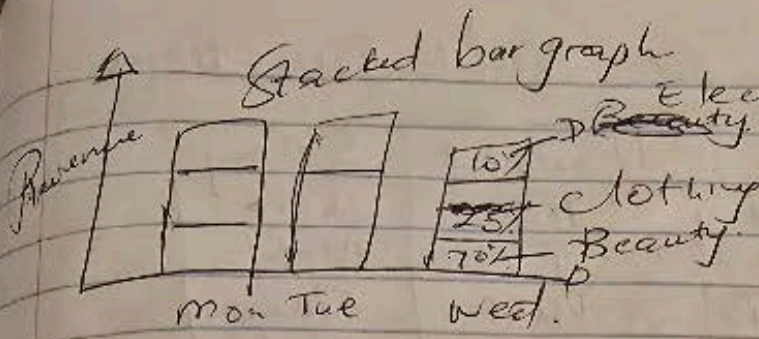
Showing ~~Proportion~~ Proportions



- Categorical Breakdown
- Metric of Contribution or proportion



Stacked bar graph



- Showing proportions for two different categorical data by a specific metric.

Product category
Beauty
Electronics
Clothing.

Day of week
mon
Tue
wed
Thur

Recap on NULL Functions

employee_id	name	department	Salary
1	Alice	HR	5000
2	Bob	IT	NULL
3	Charlie	NULL	7000
4	Dana	Finance	NULL

Syntax

```
SELECT employee_id,
       name,
       IFNULL(salary, 0) AS Salary_with_default
FROM Employees;
```

Correct

~~COALESCE (salary, default) AS salary with default~~

Correct

Table

employee_id	name	Salary_with_default
1	Alice	5000
2	Bob	0
3	Charlie	7000
4	Dana	0

Question 2.

Syntax

```
Select Employee_id
       name,
       IF NULL (department, 'Not assigned') As department
FROM Employees;
```

Employee-id	name	department_name
1	Alice	HR
2	Bob	IT
3	Charlie	Not assigned
4	Dana	Finance

Table 2

account-id	account_type	balance
A1	Savings	Null
A2	Current	5000
A3	Null	2000

Syntax

```
SELECT account-id,
    IF Null (account_type, 'unknown') AS account
                                        type,
    IF Null (balance, 0) AS balance_checked
FROM Bank accounts;
```

account-id	Account_type	Balance_checked
A1	Savings	0
A2	Current	5000
A3	unknown	2000

Syntax

~~Select *~~
~~From Reviews~~
~~Where Comment is NOT NULL AND rating~~
~~is NOT NULL~~

```
Select   review-id,
         Product-id,
         (Comment) AS comment-display,
         (rating)  AS rating-display.
WHERE    comment-display IS NOT NULL AND
         rating display IS NOT NULL
```

Wildcards

* This is a way of filtering our data using a specific pattern in a specific Column.
* You filter data based on how words are arranged on a specific column.

Wildcards Characters.

Symbol.	Description
% Persent ergin	Represent Zero or more characters
_ underscore	Represents a Single Character.

In order to use wildcards, we need to use the LIKE Statement

The LIKE Statement is commonly used in the WHERE Statement to filter data

```
SELECT *
FROM Brightlearn
WHERE name LIKE 'a%';
```

- This is an example where we are looking for Students and their name start with the letter "a"

Customer_id	Customer_name
1	John
2	Johnny
3	Joan
4	Joseph
5	Jack

① write an SQL Script to give me the names of Customers starting with letter "Jo"

```
Select*
From Customers
WHERE Customer_name LIKE 'Jo%'
```

Customer_id	Customer_name
1	John
2	Johnny
3	Joan
4	Joseph

WHERE name LIKE '%y'

Any letter can exist but the last letter of the pattern must be 'y'

Roy
Tony
Mary
Jeremy

Killd cards Day2

Syntax

```
SELECT *
FROM bright_Coffee
LIMIT 10;
```

```
SELECT transaction_date,
       Store_location
FROM bright_Coffee
WHERE Store_location LIKE 'A%';
```

ILIKE Function

```
SELECT transaction_date, Store_location
FROM bright_Coffee
WHERE Store_location ILIKE 'a%';
```

LIKE - It is case sensitive
ILIKE - It is not case sensitive

*What is case sensitive, It is values inside the table and not Headings/Column names

```
SELECT DISTINCT day_name
FROM bright_Coffee
WHERE dayname ILIKE 'S_N';
or S%
S__
```

Sum

Data Engineering Bootcamp

- Data / information - Spices.

- Knowledge - skills of knowing what spices to use

Wisdom - better ways to do it faster and quicker

- Data engineer is like a plumber. (Bobejan, tools and others)

↓

SQL Server, Visual Studio. (Built data pipelines), Git, SSMS

Operational DB MySQL / PostgreSQL

↓ Extract raw data

↓ Orchestration (Apache airflow)

Transform engine

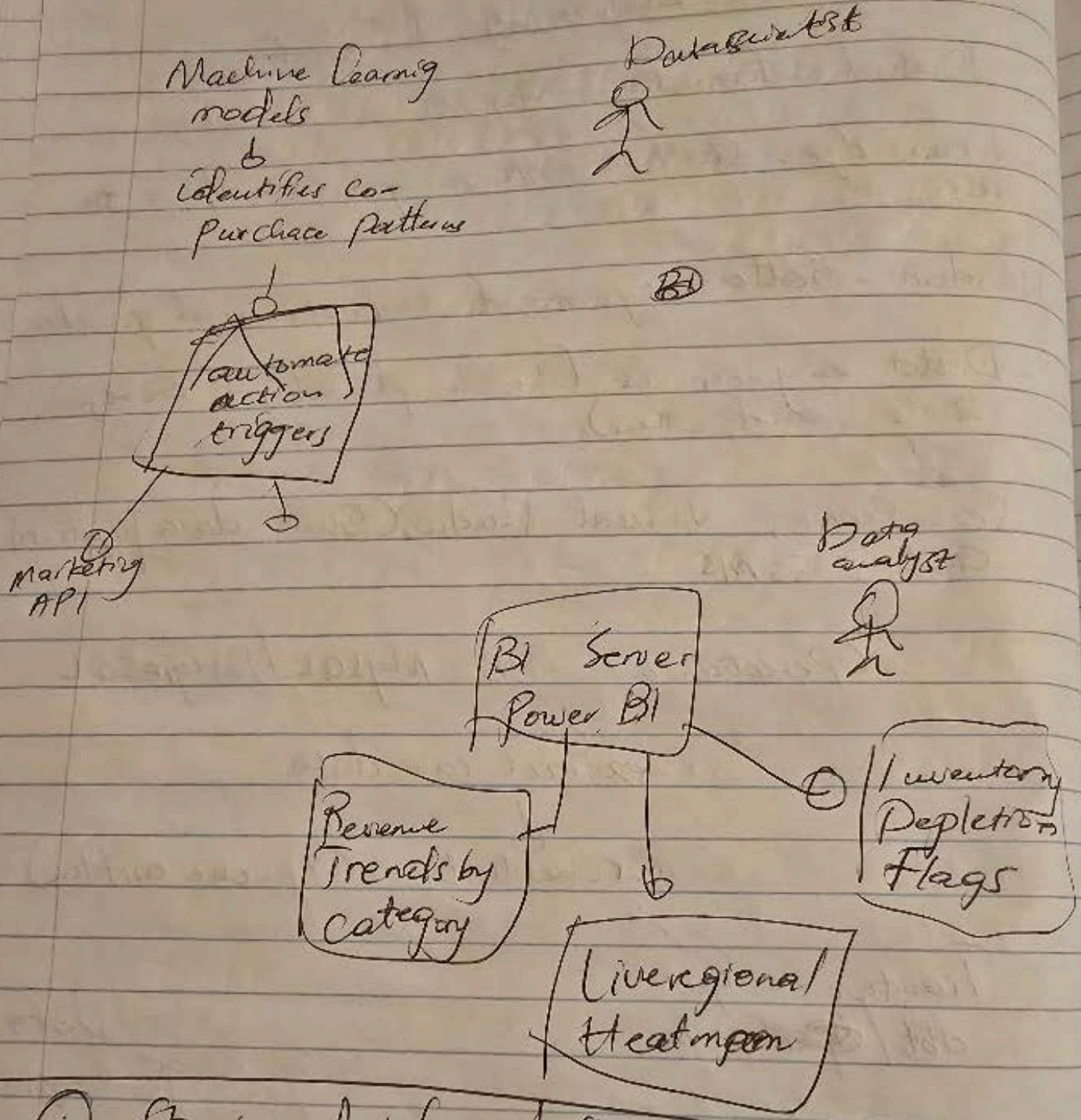
dbt / ~~Spark~~ Spark



Data Engine

↓ load clean data

↓ Cloud Data warehouse Snowflake



① Staging database | OLTP - Software, Data engineer
② data warehouse | OLAP - Data engineer, Data analyst, Data Scientist

Flat file is the CSV

Day3 Wildcards

SELECT UPPER(DAY_classification) AS DAY_CLASSIFICATION -- Converts to upper case (capital letters.
LOWER(MONTH_NAME) AS MONTH_NAME -- Converts to lower case (small letters
FROM bright_coffee;

TO DO

Canva - project plan

*project plan gchart showing project plan with ~~plan~~ timelines

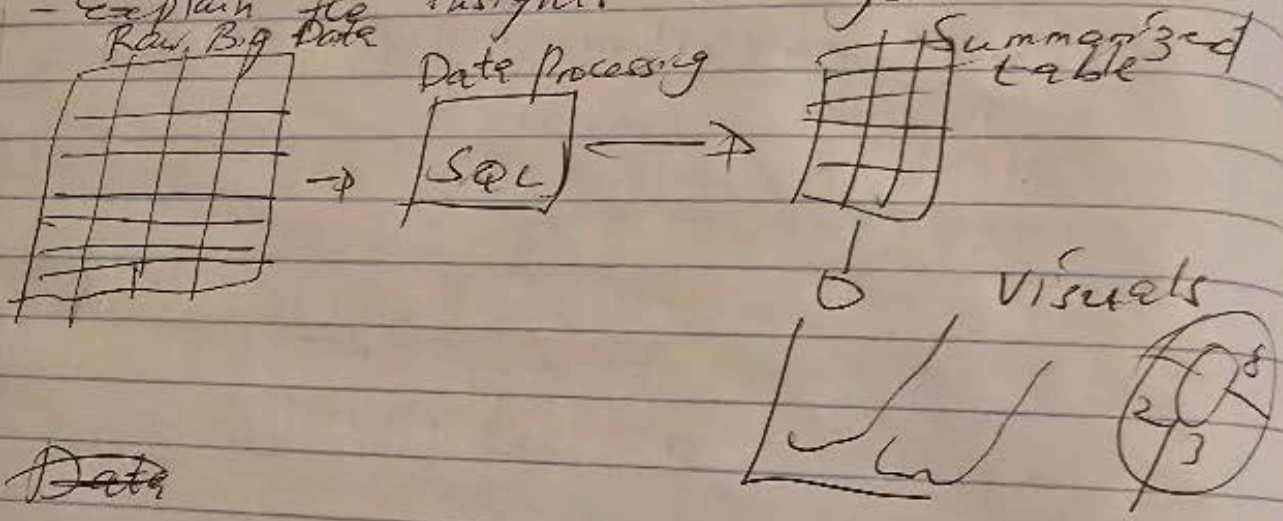
Data Visualization

↳ Data Story telling
↳ we build data visuals in order to tell the data story

↳ We build data visuals in order to extract insights from ~~visuals~~ a Complex data

- Insight - is information that I can extract from the data to use for making business dec

- We Visualize the data to see trends
- Explain the insights from your visuals.



~~Data~~

Data Analysis Process

① understand Business Questions
↳ BRS - Business Requirements

② Ask data driven questions.
↳ You want to set objectives of how you will answer business goals.

③ Data Inspection - Is the data I need to solve business available.
↳ OLAP Database

④ Exploratory Data Analysis
↳ understand the relationship of your data points.

⑤ Build Features
- Creating logical conditions that help to create new columns to enable us to tell a story.
- Creating new columns base on other columns

⑥ Build the visuals.
↳ use the right visual

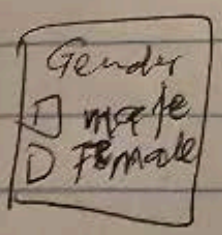
⑦ Tell the story
↳ Interpreting the visuals to highlight key actionable insights.

Building Visuals

- Categorical Data
- Numeric Data
  - Continuous Data
  - Quantitative Data

Categorical Data

We use it to build filters



Splits

Legends

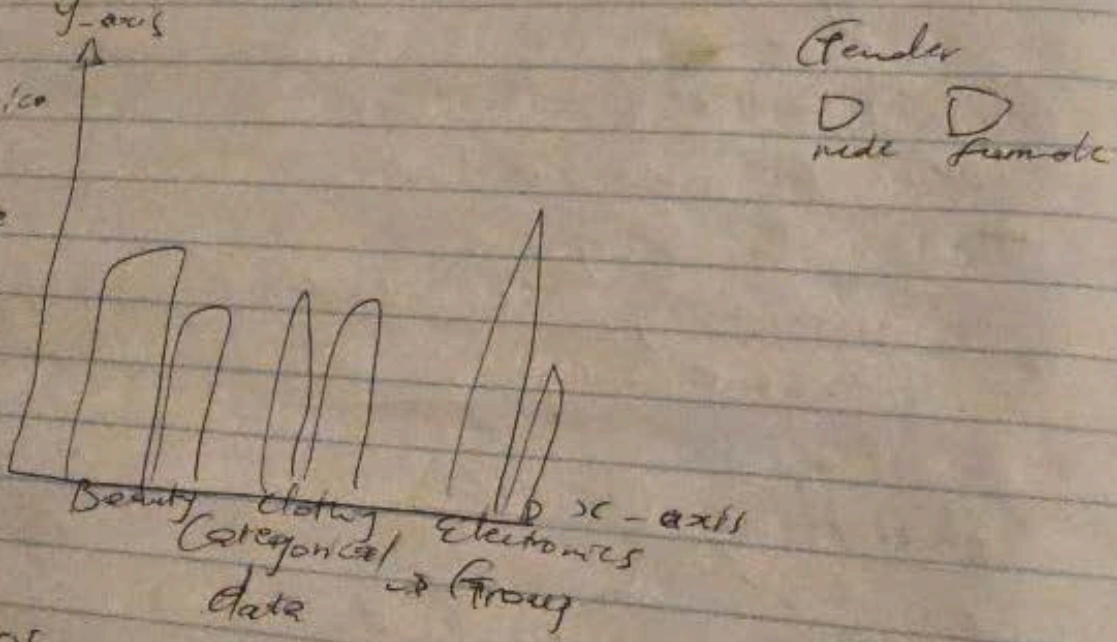
Numeric Data

We use numeric data with a card to summarize a specific KPI (key performance indicator)

Revenue (total)

Metric to measure

Bar chart



Data Story ⌐ Data visualization
⌙ Data Presentation

Page 1 - Summary page
↳ Summarize the important Measure
↳ KPIs

Page 2 - Sectional detail.
↳ → Breakdown a specific view into detailed view

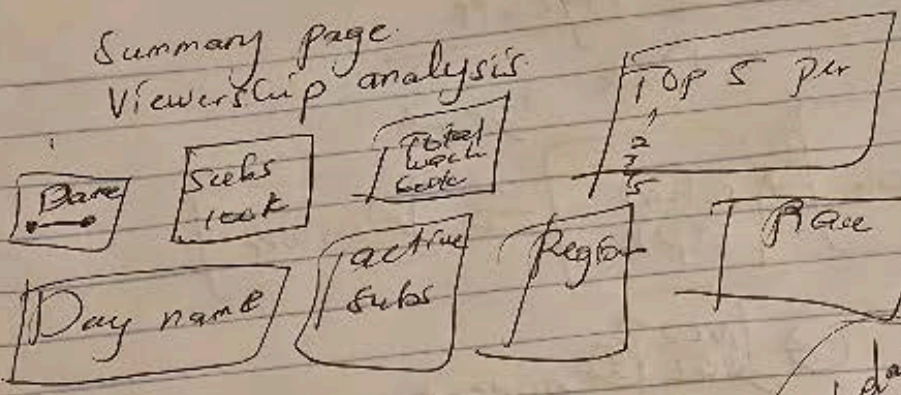
Page last : Glossary.
↳ Definitions of key words.
↳ Abbreviation → MoM.
↳ YoY
↳ MTD
Need to explain → YTD.

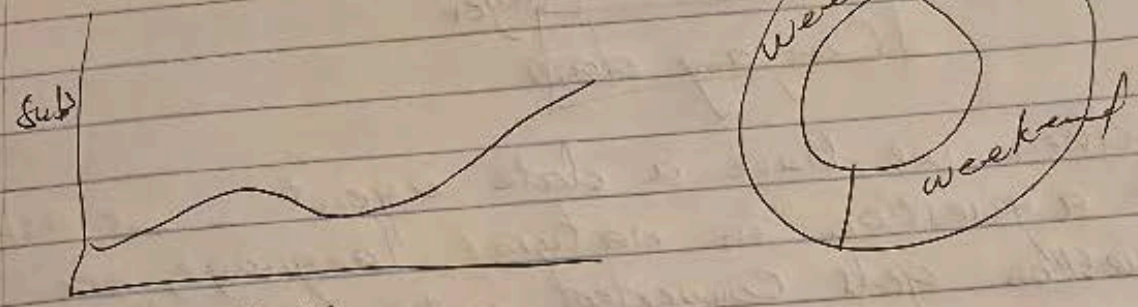
Page 0 - Cover page - explain what is in the dashboard.

Note : What is the latest data updates.

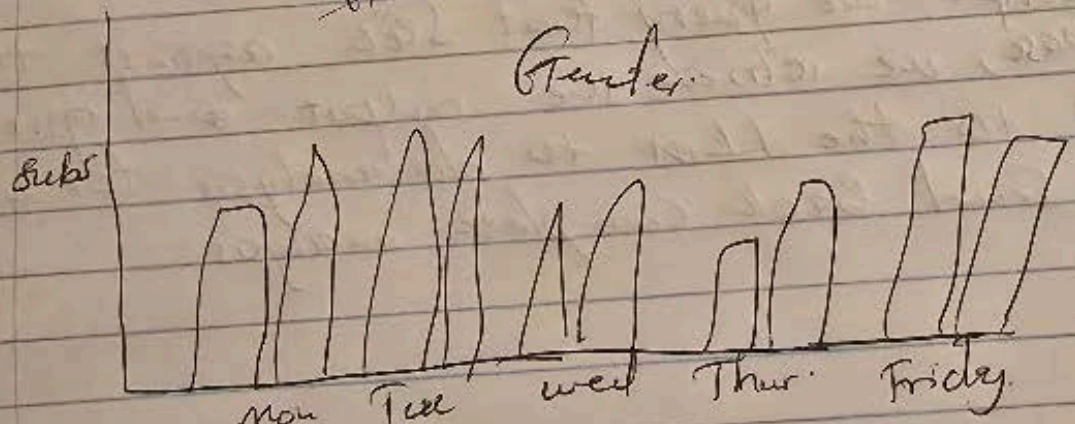
Summary page

Viewership analysis

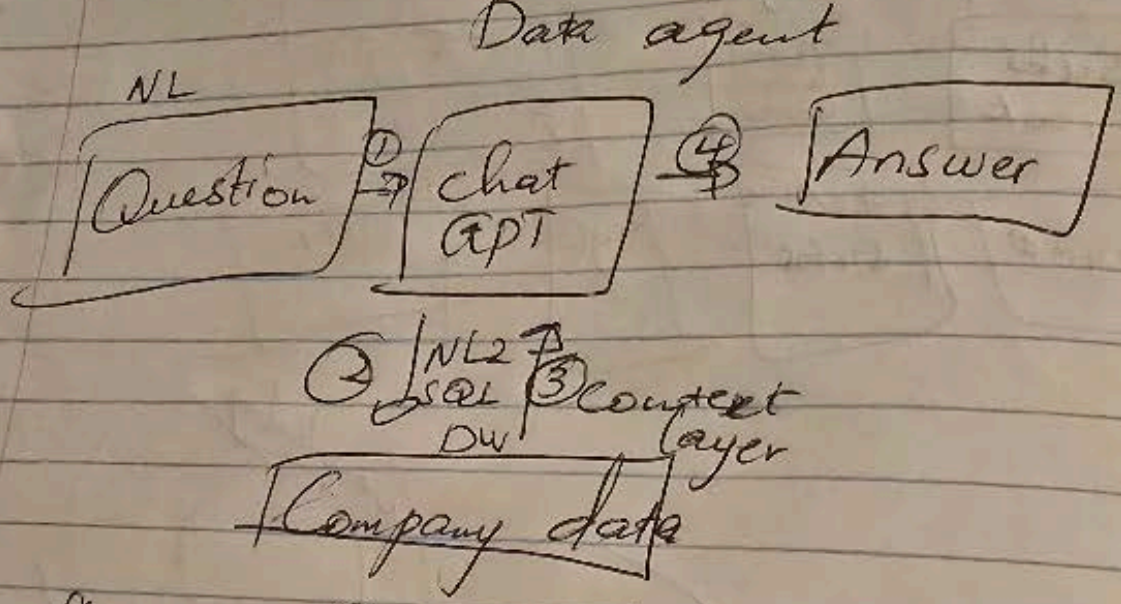




Gender



Data Agents



When we have a date agent - we ask it a question in natural language, the question gets converted into SQL - that SQL script - we query that SQL against the database, we retrieve the output and give back to the LLM to do analysis for us and send back complete analysis.